Frontend API Handoff

Product management, catalog and checkout

26 October 2026 | API contract from the current backend workspace

This guide covers the APIs implemented or updated in the recent product work: products, categories, brands, variant definitions, product variants, images, customer catalog, checkout and payment callbacks. Supporting authentication, cart, wishlist and coupon endpoints are included for integration. Lab/provider APIs and unrelated user-management modules are outside this handoff.

1. Integration setup

Use the deployment's API origin as API_ORIGIN. Local development commonly uses http://localhost:3000. Every path in this guide includes /api; do not add it twice. Example IDs, prices and hosts are illustrative, not live records or credentials.

Deployment prerequisite: migration 026-product-schema-alignment.sql is prepared and tested in an isolated database. It has NOT been applied to the configured database during the schema-alignment work. Backend deployment must stop old API writers, run npm run db:migrate, then start the updated backend. Migration 025 was applied earlier. Confirm migration 026 is deployed before integrating the new checkout flow.

Authentication and headers

Admin endpoints use the HttpOnly admin_auth cookie. Customer browser endpoints use the HttpOnly customer_auth cookie. They are separate sessions; an admin login does not authenticate customer APIs. Use credentials: "include" for browser fetch requests. Admin authentication does not accept Bearer tokens.

Browser writes, including login, PATCH and DELETE, require a trusted Origin. The browser supplies Origin automatically; its exact frontend origin must be allowed by backend configuration. Local example: http://localhost:5173. Curl must supply Origin explicitly.

Send Content-Type: application/json for ordinary writes. Send {} for actions without fields. For image POST and PUT, send FormData and let the browser set multipart/form-data and its boundary; do not set Content-Type manually. Do not attempt to read HttpOnly cookies in JavaScript.

Native customer clients send X-App-Client: native and Authorization: Bearer <customer-token>. OTP verification returns a token only when X-App-Client is native. Native requests must use JSON and must not send browser cookies. This native mode does not apply to admin routes.

```
const API_ORIGIN = 'http://localhost:3000';
const response = await fetch(`${API_ORIGIN}/api/admin/products`, {
  credentials: 'include'
});
const result = await response.json();
if (!response.ok) throw new Error(result.error || 'Request failed');
```

Shared response and validation rules

Admin create/update/detail responses normally use {data: object}; lists use {data: array, pagination: {...}}. Image and tree lists use {data: array} without pagination. Successful DELETE responses are 204 with no JSON body. Checkout create/detail responses are direct order objects, not nested under data.

Default pagination is limit=20, offset=0; limit is 1-100. Use pagination.hasMore and increment offset by the current limit. Requests use snake_case fields except checkout, address, OTP and saved-item bodies, which use their documented camelCase fields. Schemas are strict: unknown fields are rejected. PATCH must include at least one supported field. Omit unchanged values; null clears only nullable fields.

PostgreSQL BIGINT IDs and DECIMAL values may be serialized as strings. Preserve IDs as strings in frontend state; convert selected admin IDs to numbers where request schemas require JSON numeric IDs. Send prices as JSON numbers, not strings. Query parameters are strings and are coerced where documented.

```
{
  "data": [],
  "pagination": {
    "total": 42,
    "limit": 20,
    "offset": 0,
    "hasMore": true
  }
}
```

2. Admin products

All routes below require the admin cookie. Product IDs in paths are positive integers. The database columns id and name are exposed by these APIs as product_id and product_name. Do not send database field names id or name as product body fields.

Method	Full endpoint	Success
GET	/api/admin/products	200 list
GET	/api/admin/products/:productId	200 detail
POST	/api/admin/products	201 created
PATCH	/api/admin/products/:productId	200 updated
DELETE	/api/admin/products/:productId	204

List query parameters

Parameter	Values / default	Behavior
limit / offset	1-100 / >=0; defaults 20 / 0	Pagination
category_id	Positive integer	Directly assigned category; descendants are not implicitly included
brand_id	Positive integer	Filter a brand
search	Trimmed string, 1-255 chars	Name, product code, SKU, or exact keyword entry
product_status	draft, active, inactive, archived	Optional status filter
sort_by	product_id (default), product_name, product_code, sku	Allowed sort field
sort_order	asc (default), desc	Sort direction

Example: GET /api/admin/products?brand_id=3&sort_by=product_name&sort_order=asc&limit=20&offset=0

Create and edit

Required on create: product_name and slug. Slug must contain lowercase letters/digits separated by single hyphens, with no spaces. Product status defaults to draft; product type defaults to simple. category_ids defaults to []; a non-null primary_category_id must be included in the assigned categories. brand_id is optional and nullable.

```
{
  "product_name": "Adult Pull-Up Diapers",
  "slug": "adult-pull-up-diapers",
  "brand_id": 3,
  "product_type": "simple",
  "product_status": "draft",
  "short_description": "Pack of 10",
  "description": "Comfortable daily protection.",
  "product_code": "JOY-01",
  "sku": "JOY-SIMPLE",
  "regular_price": 999,
  "sale_price": 799,
  "stock_quantity": 25,
  "category_ids": [12],
  "primary_category_id": 12
}
```

For edit, fetch product detail first. Populate the form, including existing images and variant records. PATCH only supported edited fields; never POST a second product to edit an existing one. Use an edit-page title in the frontend. A status update does not remove stored images.

```
{
  "product_name": "Adult Pull-Up Diapers – Pack of 10",
  "product_status": "active"
}
```

Product list and detail responses include ordered images, variants, categories and brand. Product create/update responses return the saved base record; refetch detail when the UI needs the full related data. Images and variants are separate resources; do not submit nested images or variants in the product JSON body.

3. Product fields and pricing

Unless marked required below, these fields are optional. On PATCH, omitted fields retain their stored values. A field marked nullable can be explicitly cleared with null. Defaults shown here are for a new product.

Field(s)	Type	Contract
product_name	string	Required on create; trimmed, 1-255 chars
slug	string	Required on create; 1-280 chars, lowercase hyphen slug
brand_id	integer / null	Positive numeric brand ID
description / short_description	string / null	Trimmed text
product_status	enum	draft by default; active, inactive, archived also accepted
category_ids	integer[]	Positive category IDs; default []
primary_category_id	integer / null	Must belong to category_ids; on standalone PATCH must already be assigned
product_code / sku	string / null	Trimmed 1-100 chars; unique when provided
product_type	enum	simple by default; variable supports per-size records
regular_price / sale_price	number / null	Nonnegative currency amounts, <=999999999.99, at most 2 decimal places
mrp / offer_price	input aliases	mrp maps to regular_price; offer_price maps to sale_price; mrp is non-null
offer_percent	number / null	0-100, at most 2 decimals; backend calculates sale_price
special_discount_enabled	boolean	Default false
special_discount_percent	number	0-100, at most 2 decimals; default 0
festival_offer_enabled	boolean	Default false
festival_offer_percent	number	0-100, at most 2 decimals; default 0
stock_quantity	integer	>=0; default 0
min_order_quantity	integer	>=1; default 1
max_order_quantity	integer / null	>=1 and >= min_order_quantity; null means no product-specific maximum

Sale price requires MRP and cannot exceed it. When offer_percent is set, sale_price is recalculated from MRP, including after an MRP change. Set offer_percent to null to return to a manually supplied sale_price. Conflicting values supplied through an alias and its canonical field are rejected.

The returned effective_price starts from sale_price, or MRP if no sale price exists. Enabled special discount is applied first, then enabled festival discount, rounding to paise after each step. Example: MRP 100, sale price 80, special 10%, festival 5% produces 68.40. Display the server result; do not persist a frontend-calculated price. GST is stored as metadata and is not added by these APIs. Discount flags apply while enabled; there is no date-scheduling endpoint.

Specifications accepted on create and PATCH

Field(s)	Type	Contract
target_audience	string / null	Trimmed text, maximum 2000 chars
hsn_code	string / null	Maximum 20 chars
gst_rate	number / null	0-100, at most 2 decimals
weight	number / null	Positive; paired with weight_unit
weight_unit	enum	g by default; kg also accepted
unit_count	integer / null	Positive number of units in one pack
package_length / package_breadth / package_height	number / null	Positive dimensions
dimension_unit	enum	cm by default; mm or in also accepted
material / colour / package_type	string / null	Trimmed text, maximum 2000 chars each
shelf_life	string / null	Text such as "24 months", maximum 2000 chars
water_resistant	boolean / null	Use a boolean, not "Y" or "N" strings
hazardous_clause / manufactured_in	string / null	Maximum 2000 chars each
company_name / company_address	string / null	Maximum 2000 chars each
packer_name / packer_address	string / null	Maximum 2000 chars each
keywords	string[]	At most 100 separate strings, 1-100 chars each; default []

4. Category hierarchy

All category routes require admin authentication. Category IDs are positive integers. Responses expose category_id and category_name. category_image is a string URL/reference field; there is no category multipart upload endpoint in this module.

Method	Full endpoint	Success
GET	/api/admin/categories	200 paginated list
GET	/api/admin/categories/tree	200 nested roots
GET	/api/admin/categories/:categoryId	200 detail
POST	/api/admin/categories	201
PATCH	/api/admin/categories/:categoryId	200
DELETE	/api/admin/categories/:categoryId	204

List queries: limit, offset, search (1-150 chars), is_active=true|false, roots_only=true|false, parent_category_id=<positive integer>. Do not combine roots_only=true with parent_category_id. The tree endpoint accepts only is_active=true|false; no pagination query is supported there.

```
{
  "category_name": "Adult Care",
  "slug": "adult-care",
  "parent_category_id": 2,
  "category_image": "https://cdn.example.com/adult-care.png",
  "description": "Daily adult care",
  "sort_order": 1,
  "is_active": true
}
```

Create requires category_name (1-150 chars) and slug (1-180 chars). Other fields are optional. parent_category_id, category_image and description are nullable. sort_order defaults to 0 and is_active to true. PATCH accepts any nonempty subset. Set parent_category_id=null to make a root category. Self-parenting and hierarchy cycles are rejected.

Recommended UI sequence: load GET /categories?roots_only=true, then fetch children with parent_category_id after selection. Fetch products for that selected category with GET /api/admin/products?category_id=12. Alternatively, load /categories/tree once and render its nested subcategories arrays. The frontend controls expansion and when items become visible.

Tree response shape (illustrative):

```
{
  "data": [{
    "category_id": "2",
    "category_name": "Care",
    "parent_category_id": null,
    "subcategories": [{
      "category_id": "12",
      "category_name": "Adult Care",
      "parent_category_id": "2",
      "subcategories": []
    }]
  }]
}
```

Deletion returns 409 while child categories or product assignments exist. Remove/reassign them first.

5. Brands

All brand routes require admin authentication. API fields use brand_id, brand_name and image_url. The backend maps image_url to the database logo_url column; the frontend should continue sending image_url.

Method	Full endpoint	Success
GET	/api/admin/brands	200 list
GET	/api/admin/brands/:brandId	200 detail
POST	/api/admin/brands	201
PATCH	/api/admin/brands/:brandId	200
DELETE	/api/admin/brands/:brandId	204

List queries: limit, offset, search (1-150 chars), is_active=true|false, all=true|false. Use all=true for the complete brand selector; it ignores pagination and returns hasMore=false. Without all=true, follow pagination.hasMore rather than showing only the first page. Ordering is brand_name then brand_id.

```
{
  "brand_name": "Joylife",
  "slug": "joylife",
  "description": "Adult care products",
  "image_url": "https://cdn.example.com/joylife.png",
  "is_active": true
}
```

Create requires brand_name (1-150 chars) and slug (1-180 chars, lowercase hyphen pattern). description is optional nullable text up to 10000 chars. image_url is an optional nullable valid URL. is_active is optional and defaults to true. PATCH accepts any nonempty subset.

Deletion returns 409 when products still reference the brand. Reassign those products or PATCH brand_id:null before deleting. To clear a product's brand, use the product PATCH endpoint, not a blank string or brand ID 0.

6. Variant attribute definitions

An attribute definition lists separate allowable options. A product variant selects exactly one option for each attribute it uses. These resources are different: creating a "Size" definition does not create any product variants.

Method	Full endpoint	Success
GET	/api/admin/variant-attributes	200 paginated list
POST	/api/admin/variant-attributes	201
PATCH	/api/admin/variant-attributes/:attributeId	200
DELETE	/api/admin/variant-attributes/:attributeId	204

GET supports limit and offset and sorts by name then ID. There is no single-attribute GET endpoint; obtain the definition from the list. Definitions expose id, name and values.

```
{
  "name": "Size",
  "values": ["M", "L", "XL", "XXL"]
}
```

Create requires name and values. Name and each option are trimmed strings of 1-100 chars. There must be 1-100 unique options. Commas are not allowed within a name or option: send ["M","L"] rather than ["M,L"]. Display options as separate tabs, chips or inputs.

PATCH accepts name and/or the entire values array. Adding options is allowed. Renaming a definition, removing options, or deleting a definition returns 409 when it is used by variants. The guard conservatively blocks removing any old option when the definition is in use, even if that specific option was not selected.

7. Per-product variants

Set product_type="variable" on the parent before creating variants. Each size/combination is a separate record with its own ID, SKU, stock and optional prices. All routes use admin authentication and scope the variant to its parent product.

Method	Full endpoint	Success
GET	/api/admin/products/:productId/variants	200 list
GET	/api/admin/products/:productId/variants/:variantId	200 detail
POST	/api/admin/products/:productId/variants	201
PATCH	/api/admin/products/:productId/variants/:variantId	200
DELETE	/api/admin/products/:productId/variants/:variantId	204

The list accepts limit and offset and sorts by sku then id. Create requires sku and attributes. Attribute names and selected options must already exist in the definitions. PATCH accepts any nonempty subset of these fields:

Field	Type	Contract
sku	string	Required on create; 1-100 chars, unique
attributes	object	Required on create; 1-20 pairs such as {"Size":"M"}; each name/value 1-100 chars, no commas
variant_name	string / null	Optional label, 1-200 chars
barcode	string / null	Optional, 1-100 chars; preserve leading zeros
weight_grams	integer / null	0-2147483647
regular_price / sale_price	number / null	Same money limits as products; sale cannot exceed regular price
stock_quantity	integer	>=0; default 0
is_active	boolean	Default true

```
{
  "sku": "JOY-M",
  "variant_name": "Medium pack",
  "attributes": {"Size": "M"},
  "barcode": "0123456789012",
  "weight_grams": 250,
  "regular_price": 999,
  "sale_price": 799,
  "stock_quantity": 25,
  "is_active": true
}
```

Responses use {data: {...}} and expose id, product_id, all variant fields, timestamps and display_name. display_name prefers variant_name and otherwise joins selected options, for example "M / Blue". Render it beside the parent product name for quick editing.

Duplicate SKUs or duplicate complete attribute combinations return 409. Unknown definitions/options return 400. A variant that does not belong to the URL's product returns 404. Remove variants before changing a parent back to product_type="simple". In the customer catalog, a variant inherits parent pricing when its regular_price is absent; explicit variant pricing uses the parent special/festival discount flags.

8. Product image CRUD and replacement

All image routes use admin authentication. productId and imageId must be positive integers. Product JSON bodies do not accept images. Upload separately after obtaining a product_id.

Method	Full endpoint	Success
POST	/api/admin/products/:productId/images	201 array of created images
GET	/api/admin/products/:productId/images	200 array of stored images
PATCH	/api/admin/products/:productId/images/:imageId/primary	200 selected image
PUT	/api/admin/products/:productId/images/:imageId	200 replacement metadata
DELETE	/api/admin/products/:productId/images/:imageId	204

Multipart field name is uploadimage. Append it once per file. Accepted MIME types: image/jpeg, image/png, image/webp. Maximum 6 stored images per product, 5 MB per file, 10 MB total per upload request. POST with no files is 400. PUT must contain exactly one file. Unsupported MIME and invalid file counts are client errors.

```
const form = new FormData();
for (const file of selectedFiles) form.append('uploadimage', file);
const response = await fetch(
  `${API_ORIGIN}/api/admin/products/${productId}/images`, {
    method: 'POST', credentials: 'include', body: form
  }
);
const result = await response.json();
if (!response.ok) throw new Error(result.error || result.message);
```

For replacement, create a new FormData with ONE uploadimage file and send PUT to /images/{imageId}. Do not set Content-Type. Replacement preserves the image ID and storage key and refreshes image_url with a version query parameter. Store/display the returned URL; do not reconstruct it or strip its query string.

```
{
  "data": [{
    "id": "44",
    "product_id": "120",
    "image_url": "https://example-bucket.s3.example.com/upload/120a.png",
    "s3_key": "upload/120a.png",
    "alt_text": null,
    "sort_order": 0,
    "is_primary": true
  }]
}
```

GET orders images by primary first, then sort_order, then ID. Changing primary uses PATCH with JSON {}. Deleting a primary promotes the next image. Refetch the list after primary selection or deletion. Product list/detail return images as well; use them to repopulate the edit form. Publishing preserves uploaded images. No endpoint currently edits alt_text or reorders sort_order independently.

9. Customer product catalog

These endpoints require customer authentication, not the admin cookie. Products must be published and priced to appear. The list is database-backed; it no longer returns the keyboard/mouse demo data.

Method	Full endpoint	Success
GET	/api/app/products	200 paginated list
GET	/api/app/products/:id	200 detail; 404 if unavailable

List queries: limit (default 20, max 100), offset (default 0), search (name, 1-255 chars), category_id (string, 1-100 chars). Category filtering supports the catalog's category_ids array or its categoryId field.

Catalog IDs are strings: product:120 for a simple product, variant:81 for a variant, or a retained seeded ID such as p1. Treat IDs as opaque and use encodeURIComponent when inserting them into URLs. Do not use an admin product_id or variant_id as the checkout item ID. Read the id returned by this catalog endpoint.

An illustrative detail response, with additional specification metadata omitted:

```
{
  "data": {
    "id": "variant:81",
    "name": "Adult Pull-Up Diapers – Medium pack",
    "price": 799,
    "mrp": 999,
    "stock_quantity": 25,
    "categoryId": "12",
    "category_ids": ["12"],
    "image": "https://cdn.example.com/product.png",
    "images": [],
    "attributes": {"Size": "M"},
    "variant_name": "Medium pack"
  }
}
```

Admin product, variant, category-assignment and image saves refresh catalog snapshots transactionally. Draft/inactive/deleted/unpriced entries become unavailable. An existing admin product is published when next saved; there is no automatic bulk publication of all older products. Display catalog price as authoritative; do not recalculate it from MRP. Stock 0 can still appear in the list; disable purchase controls using stock_quantity and show checkout errors if stock changes.

10. Checkout and order history

All three routes require customer authentication. Route names stay /orders even though persistence now uses product_checkouts. Do not call the database table name as an API route.

Method	Full endpoint	Success
POST	/api/app/orders	201 ready payment session; 200 reused checkout; 202 pending/uncertain session
GET	/api/app/orders	200 {data: orders[]}
GET	/api/app/orders/:id	200 direct order object after payment verification

Create requires requestKey, items and address. The customer's profile must have both email and phone or checkout returns 422. Do not submit totals, discounts or unit prices; the backend reads current catalog prices.

```
{
  "requestKey": "checkout-26oct2026-unique-0001",
  "items": [{"id": "variant:81", "quantity": 2}],
  "address": {
    "line1": "Flat 12, Test Road",
    "line2": "Baner",
    "landmark": "Near Main Gate",
    "city": "Pune",
    "state": "Maharashtra",
    "postalCode": "411045"
  },
  "notes": "Call before delivery",
  "couponCode": "SAVE10"
}
```

requestKey is a string of 10-100 chars. Generate it once for a checkout attempt and retain it across retries. items must contain 1-50 unique IDs, each 1-100 chars, with integer quantity 1-99. Product-specific min/max quantities are also enforced. notes defaults to an empty string and is at most 1000 chars. couponCode is optional, normalized uppercase, and at most 40 chars containing letters, digits, underscore or hyphen. Omit it when not used. The legacy coupon field accepts only false; it is not a discount toggle.

Address contract

line1: 5-200 trimmed chars; city/state: 2-100 chars each; postalCode: six digits with a nonzero first digit. Optional line2 and landmark are up to 200 chars each and default to "". Optional houseNumber (up to 100 chars) and buildingName (up to 200 chars) also default to "".

Optional location is null or an object containing provider="locationiq", placeId (up to 200 chars), displayName (1-2000 chars), latitude (-85 to 85), longitude (-180 to 180), and address (object of string values). If location is supplied, houseNumber must not be empty. This checkout address schema permits omitted location; saved-address endpoints have stricter locality requirements.

Checkout response and frontend behavior

Illustrative response (partial; the real object also contains snapshots and delivery details):

```
{
  "id": "MB0123456789abcdef01",
  "orderNumber": "MB26102600001",
  "kind": "products",
  "status": "Payment pending",
  "paymentStatus": "Pending",
  "subtotal": 1598,
  "discount": 0,
  "delivery": 0,
  "total": 1598,
  "stockState": "reserved",
  "paymentExpiry": null,
  "paymentUrl": "https://smartgateway.hdfcuat.bank.in/payment-page/example"
}
```

id is the gateway-facing ID (MB plus 18 lowercase hex characters); use it in GET /orders/:id. orderNumber is a separate customer-facing 13-character MBYYMMDDnnnnn number based on the India date. Never put orderNumber into the :id route.

Open a non-null paymentUrl in hosted checkout. On return or app foreground, call GET /orders/:id and render the server's verified paymentStatus. A browser redirect or bank-supplied URL parameter is not proof of payment. 202 means the checkout is saved but the payment session is not confirmed; retain the order ID, show a pending/recovery state and check payment status instead of starting a second payment.

Retrying the same request key with different details returns 409. Identical unpaid checkout details may reuse the existing checkout even with a new key. GET /orders lists saved snapshots and current saved status; it does not query the bank for every item. GET /orders/:id performs status verification. Paid checkouts return paymentUrl=null; stored expired links are also suppressed.

Delivery is 40 below a subtotal of 499 and 0 from 499 upward, before coupon discount. All displayed totals come from the response. Managed inventory is reserved once at checkout and committed after verified payment. Stock states are legacy, unmanaged, reserved and committed. Failed/uncertain payments retain reservations for operational reconciliation; no refund, cancellation or automatic stock-release endpoint is implemented.

11. Payment callbacks and supporting shopping APIs

Payment callbacks are backend/gateway integrations, not normal authenticated frontend requests.

Method	Full endpoint	Purpose
POST	/api/payments/hdfc/webhook	Gateway-only; HTTP Basic credentials; body content.order.order_id; successful acknowledgements are 200
GET	/api/payments/hdfc/return	200 HTML page linking back to the app
POST	/api/payments/hdfc/return	200 HTML; form-urlencoded return supported

Never embed webhook credentials or merchant API secrets in the frontend. The return page asks the user to return to the app and check status. Payment verification uses server-to-server bank requests with merchant, order, currency and amount checks.

Cart, wishlist and coupon support

These existing customer endpoints support the new catalog IDs. They use customer authentication.

Method	Full endpoint	Success
GET	/api/app/cart	200 {data: [{id, quantity}]}
PUT	/api/app/cart/items/:id	200 {id, quantity}; JSON {"quantity": 2}
DELETE	/api/app/cart/items/:id	204; JSON {}
GET	/api/app/wishlist	200 {data: [{id}]}
PUT	/api/app/wishlist/items/:id	200 {id}; JSON {}
DELETE	/api/app/wishlist/items/:id	204; JSON {}
GET	/api/app/coupons?scope=products	200 {data: active coupon suggestions}

Saved-item IDs are strings up to 100 chars. Cart quantities are integers 1-99. PUT verifies that the catalog entry is still active; unavailable entries return 422. Each collection is limited to 200 distinct entries. Existing redcliffe: lab IDs are also supported by the saved-item service, with quantity fixed at 1; they are not product-checkout IDs. Coupon list scope accepts products or labs; use products for this flow. Coupons with stored scope=both are included in either applicable list. Coupons return code, scope, percent and max_discount_paise. max_discount_paise is in paise; do not present it as rupees without conversion.

12. Authentication endpoint reference

These are supporting authentication routes, included so the team can establish the sessions required above. Ordinary browser writes still require trusted Origin and JSON. Admin and customer accounts use different login mechanisms.

Method	Full endpoint	Request / result
POST	/api/admin/auth/login	{email, password}; 200 user + admin cookie
GET	/api/admin/auth/me	200 {user: {id, name, email}}
POST	/api/admin/auth/logout	JSON {}; 204
POST	/api/admin/auth/logout-all	JSON {}; 204
POST	/api/app/auth/otp/request	{method: "phone" or "email", contact: "..."}
POST	/api/app/auth/otp/verify	{challengeId, code}; six-digit code string
GET	/api/app/auth/me	Current customer account response
POST	/api/app/auth/logout	JSON {}; end current session
POST	/api/app/auth/logout-all	JSON {}; end all customer sessions

OTP request returns challengeId, expiresIn, resendAfter and delivery. Verification returns the customer account and establishes the session. Admin login passwords must be at least 12 characters and at most 72 UTF-8 bytes. OTP verification optionally accepts deviceInfo with required platform (max 30 chars), optional osVersion (max 100 chars), and optional model (max 100 chars). Reuse the challengeId returned by OTP request. Do not guess an OTP or hardcode credentials. OTP delivery depends on backend configuration; request/verification can return configured delivery and verification errors.

13. Error handling and frontend checklist

HTTP	Meaning	Frontend handling
400	Invalid relationship, image MIME/count/size, or cross-field business input	Display the returned error; keep the form state
401	Missing/expired/wrong-realm authentication	Restore the appropriate login flow
403	Untrusted browser Origin	Check frontend origin and backend CORS/Origin configuration
404	Missing, inactive or wrong-parent resource	Refetch the parent/list; avoid assuming the ID still exists
409	Duplicate record, resource in use, insufficient stock, changed checkout details	Display conflict; refetch affected data before retrying
415	Wrong Content-Type	JSON for ordinary writes; FormData for image POST/PUT
422	Schema validation, unavailable product, invalid coupon/profile/address	Show details near fields where available
429	Rate limit	Back off and respect retry information
500	Unexpected backend failure	Show a recoverable error; retain checkout IDs and user input
502 / 503	Provider failure, unconfigured payments, or daily capacity reached	Preserve saved checkout state and avoid duplicate payment submissions

```
{
  "error": "Validation failed",
  "details": [{
    "path": "primary_category_id",
    "message": "primary_category_id must be included in category_ids"
  }]
}
```

Other errors normally use {"error":"message"}; some image-handler validation responses can use {"message":"message"}. Use error || message as the fallback. Validation paths may be empty for whole-body rules. Do not attempt response.json() on 204. Successful 202 responses are pending states, not failures.

Implementation sequence: authenticate; load category and brand selectors; create or fetch the product; save product fields; upload images; define separate variant options; create per-size variants; publish the product; verify customer catalog; use returned catalog IDs for saved items and checkout; check payment status after hosted payment.

Integration checks: complete brand pagination or all=true; category expansion after root selection; existing image previews on edit; PUT replacement with one file; clear nullable selections with null; use returned image URLs; keep variant SKUs distinct; preserve checkout requestKey during retries; use gateway id for status routes; trust server totals and verified payment state.

Validation status: catalog/image and schema/checkout integration tests passed in disposable PostgreSQL databases. Earlier live curl tests verified real S3 upload, read, replacement and deletion. No live bank payments were made. The repository-wide test suite retains one unrelated Redcliffe catalog-pagination failure. This guide describes current source behavior; it does not assert that migration 026 has been deployed.